



DOCUMENTACIÓN TÉCNICA

Proyecto:
KEISY MEDICAL
Versión 1.3
Estado: Final

Autor(a): Emily Monterrosa
Instructor: Ing. Fabián Florian

Competencia: Desarrollo de Solución de Software

Servicio Nacional de Aprendizaje - SENA

Barranquilla, Colombia - 22 junio 2026

Índice de Contenido

1. CODIFICACIÓN	4
1.1 Descripción General del Código Fuente.....	4
1.2 Estructura del repositorio GitHub	6
1.3 Principales Módulos Implementados.....	6
1.4 Repositorio GitHub	7
1.5 Scripts SQL	8
2. EVIDENCIAS DEL PROCESO ETL	9
2.1 Descripción del proceso ETL	9
2.2 Evidencias de ejecución	11
2.3 Validación de datos.....	11
2.4 Reportes Generados.....	12
3. DOCUMENTACIÓN TÉCNICA	14
3.1 Descripción de la arquitectura.....	14
3.2 Despliegue en la Nube.....	14
3.3 Procedimiento de Despliegue	15
3.4 APIs desarrolladas.....	16
4. MANUAL DE USUARIO.....	17
4.1 Proceso ETL para el Usuario	17
4.2 Consulta de Reportes	17
4.3 Interpretación de Resultados	18
5. DIAGRAMAS	19
5.1 Arquitectura del software	19
5.2 Flujo ETL (Extract, Transform, Load)	22
5.3 Modelo Entidad-Relación (ER).....	25
6. EVIDENCIA MACHINE LEARNING	28
6.1 Objetivo del Modelo	28
6.2 Dataset Utilizado.....	28
6.3 Preprocesamiento.....	30
6.4 Algoritmo Utilizado	31
6.5 Entrenamiento	32
6.6 Métricas Obtenidas	33
6.7 Resultados.....	34
6.8 Conclusiones	35

7. CONCLUSIÓN	37
8. RECOMENDACIONES O TRABAJO FUTURO	38
8.1 Mejoras en el Proceso ETL.....	38
8.2 Evolución del Modelo de Machine Learning.....	39
8.3 Mejoras en Reportes y Visualización	39
8.4 Seguridad y Gobierno de Datos	39
8.5 Escalabilidad y Arquitectura Cloud.....	39
8.6 Nuevas Funcionalidades.....	40

1. CODIFICACIÓN

1.1 Descripción General del Código Fuente

Objetivo del Sistema

El proyecto *KEISY MEDICAL* es un sistema web desarrollado para la gestión, procesamiento y análisis de datos clínicos mediante procesos ETL (Extract, Transform, Load) y técnicas de Machine Learning.

El sistema permite cargar datasets médicos en multiformato (CSV, XLSX, JSON), realizar procesos automáticos de limpieza y transformación de datos, calcular indicadores clínicos como el Índice de Masa Corporal (IMC), clasificar niveles de riesgo de pacientes y almacenar la información procesada en una base de datos PostgreSQL. Además, incorpora un módulo de Machine Learning para entrenar modelos predictivos y generar métricas de desempeño que apoyen la toma de decisiones.

Objetivos específicos

- Automatizar la carga y procesamiento de datos clínicos.
- Garantizar la calidad de los datos mediante validaciones y transformaciones.
- Centralizar la información médica en una base de datos estructurada.
- Visualizar indicadores clave mediante dashboards interactivos.
- Implementar modelos predictivos para clasificación de riesgo clínico.
- Facilitar la trazabilidad mediante auditoría y registro de procesos ETL.

Tecnologías Utilizadas

Backend

Tecnología	Descripción
Python 3	Lenguaje principal del proyecto
Django	Framework web principal
Django REST Framework	Desarrollo de APIs REST
JWT (SimpleJWT)	Autenticación basada en tokens
Gunicorn	Servidor WSGI para despliegue

Base de Datos

Tecnología	Descripción
PostgreSQL	Base de datos relacional
Supabase	Servicio cloud para PostgreSQL
Transaction Pooler	Optimización de conexiones

Procesamiento de Datos

Tecnología	Descripción
Pandas	Manipulación y transformación de datos
NumPy	Operaciones matemáticas y estadísticas

Machine Learning

Tecnología	Descripción
ScikitLearn	Entrenamiento y evaluación de modelos
Random Forest Classifier	Modelo de clasificación implementado

Frontend

Tecnología	Descripción
HTML5	Estructura de interfaces
CSS3	Estilos visuales
JavaScript	Interactividad
Bootstrap 5	Framework de diseño responsivo
Chart.js	Visualización de métricas y estadísticas

Documentación y Desarrollo

Tecnología	Descripción
Git	Control de versiones
GitHub	Repositorio del proyecto
Render	Plataforma de despliegue

Estructura General del Proyecto

El proyecto está organizado bajo una arquitectura modular basada en aplicaciones Django, donde cada módulo tiene una responsabilidad específica.

```

keisy/
├── keisy/
│   ├── settings.py
│   ├── urls.py
│   ├── wsgi.py
│   └── asgi.py
├── authentication/
│   ├── models.py
│   ├── views.py
│   └── urls.py
├── patients/
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   └── urls.py
├── etl/
│   ├── services.py
│   ├── views.py
│   └── urls.py
├── dashboard/
│   ├── views.py
│   └── urls.py
├── ml/
│   ├── trainer.py
│   ├── predictor.py
│   └── models.py
├── reports/
├── templates/
│   ├── assets/
│   ├── scripts/
│   ├── styles/
│   ├── views/
│   └── index.html
├── manage.py
├── requirements.txt
├── LICENSE
└── README.md
    
```

1.2 Estructura del repositorio GitHub

Carpeta	Función
authentication	Gestión de usuarios, autenticación y permisos
patients	Administración y almacenamiento de información clínica
etl	Procesos de extracción, transformación y carga de datos
dashboard	Visualización de KPIs, métricas y estadísticas
ml	Entrenamiento y evaluación de modelos predictivos
reports	Generación de reportes y exportaciones
templates	Interfaces gráficas del sistema
keisy	Configuración principal del proyecto Django

1.3 Principales Módulos Implementados

Módulo ETL (Extract, Transform, Load)

El módulo ETL es el componente encargado de la integración y procesamiento de datos clínicos provenientes de archivos CSV. Su función principal es garantizar que la información ingresada al sistema sea consistente, válida y esté preparada para su posterior análisis.

Funcionalidades implementadas

- Carga de archivos CSV mediante interfaz web.
- Extracción de datos utilizando Pandas.
- Validación de estructura y columnas requeridas.
- Eliminación de registros duplicados.
- Conversión automática de tipos de datos.
- Gestión de valores nulos.
- Normalización de variables clínicas.
- Cálculo del Índice de Masa Corporal (IMC).
- Clasificación automática del nivel de riesgo del paciente.
- Almacenamiento de registros procesados en PostgreSQL.
- Registro de ejecuciones ETL para auditoría y trazabilidad.

Módulo de Reportes

El módulo de reportes proporciona una visión consolidada de la información procesada, permitiendo a los usuarios consultar indicadores clínicos y resultados operativos del sistema.

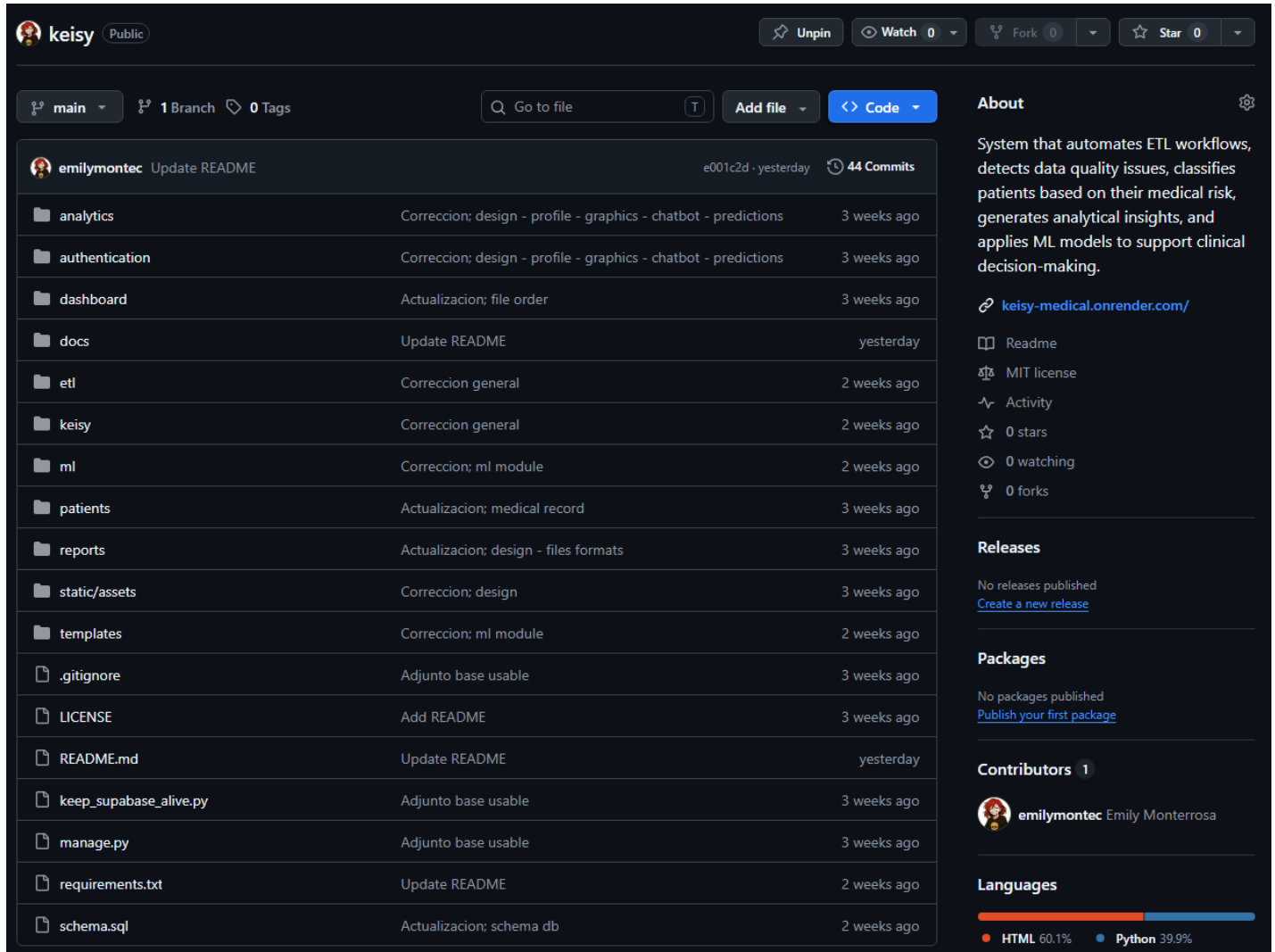
Funcionalidades implementadas

- Visualización de indicadores clave (KPIs).
- Resumen estadístico de pacientes.
- Distribución por niveles de riesgo.
- Resumen de ejecuciones ETL.
- Visualización de métricas del modelo de Machine Learning.
- Presentación de información mediante gráficos interactivos.
- Indicadores disponibles
- Total de pacientes registrados.
- Pacientes por nivel de riesgo.
- Cantidad de datasets procesados.
- Pacientes críticos identificados.
- Métricas de desempeño del modelo predictivo.
- Distribución de variables clínicas.

1.4 Repositorio GitHub

URL del repositorio

github.com/emilymontec/keisy



keisy Public

Unpin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file Add file Code

emilymontec Update README e001c2d · yesterday 44 Commits

File/Folder	Description	Commit Date
analytics	Correccion; design - profile - graphics - chatbot - predictions	3 weeks ago
authentication	Correccion; design - profile - graphics - chatbot - predictions	3 weeks ago
dashboard	Actualizacion; file order	3 weeks ago
docs	Update README	yesterday
etl	Correccion general	2 weeks ago
keisy	Correccion general	2 weeks ago
ml	Correccion; ml module	2 weeks ago
patients	Actualizacion; medical record	3 weeks ago
reports	Actualizacion; design - files formats	3 weeks ago
static/assets	Correccion; design	3 weeks ago
templates	Correccion; ml module	2 weeks ago
.gitignore	Adjunto base usable	3 weeks ago
LICENSE	Add README	3 weeks ago
README.md	Update README	yesterday
keep_supabase_alive.py	Adjunto base usable	3 weeks ago
manage.py	Adjunto base usable	3 weeks ago
requirements.txt	Update README	2 weeks ago
schema.sql	Actualizacion; schema db	2 weeks ago

About

System that automates ETL workflows, detects data quality issues, classifies patients based on their medical risk, generates analytical insights, and applies ML models to support clinical decision-making.

keisy-medical.onrender.com/

Readme MIT license Activity 0 stars 0 watching 0 forks

Releases

No releases published [Create a new release](#)

Packages

No packages published [Publish your first package](#)

Contributors 1

emilymontec Emily Monterrosa

Languages

HTML 60.1% Python 39.9%

Figura 1. Repositorio público en GitHub.

Ramas utilizadas

Durante el desarrollo del proyecto se utilizó una estrategia de control de versiones simplificada basada en una única rama principal denominada main.

Rama Principal (main)

La rama *main* fue utilizada como rama principal de desarrollo y despliegue del proyecto. Todas las funcionalidades implementadas, correcciones de errores y mejoras fueron integradas directamente en esta rama debido al alcance y tamaño del proyecto.

Características de la estrategia utilizada

- Desarrollo centralizado en una única rama.
- Gestión simplificada del control de versiones.
- Menor complejidad en la integración de cambios.
- Adecuada para proyectos académicos y MVP (Minimum Viable Product).
- Facilita el despliegue continuo al mantener una única versión estable del sistema.

Flujo de trabajo aplicado**Justificación**

La utilización exclusiva de la rama *main* permitió concentrar el esfuerzo en la implementación de las funcionalidades principales del sistema sin la necesidad de administrar múltiples ramas de trabajo. Esta estrategia resultó adecuada considerando que el proyecto fue desarrollado por un único integrante y que el objetivo principal era construir un producto funcional que integrara procesos ETL, visualización de datos, APIs REST y modelos de Machine Learning.

Recomendación para futuras versiones

Para futuras iteraciones o desarrollos colaborativos, se recomienda adoptar una estrategia basada en ramas, por ejemplo:

main: versión estable en producción.

develop: integración de nuevas funcionalidades.

*feature/**: desarrollo de características específicas.

*hotfix/**: corrección de errores críticos.

1.5 Scripts SQL**Script (accede al siguiente enlace para obtener el script completo)**

[schema.sql](#)

Objetivo

Los scripts SQL implementados en el proyecto tienen como finalidad crear y administrar la estructura de almacenamiento necesaria para soportar los procesos ETL, la gestión de pacientes, la auditoría de cambios y el almacenamiento de resultados generados por el módulo de Machine Learning.

Estos scripts permiten:

- Crear las tablas principales del sistema.
- Definir relaciones entre entidades.
- Garantizar la integridad de los datos mediante claves primarias y foráneas.
- Facilitar el almacenamiento de información clínica procesada.
- Mantener trazabilidad de las ejecuciones ETL y modificaciones realizadas por los usuarios.

Tablas Afectadas

Tablas	Descripción
patients_patient	Almacena la información clínica de los pacientes.
etl_uploadeddataset	Registra las ejecuciones del proceso ETL.
patients_patientaudit	Permite auditar cambios realizados sobre los registros clínicos.
auth_user	Tabla estándar de Django para autenticación y gestión de usuarios.

Relación con el Proyecto

Los scripts SQL son ejecutados sobre PostgreSQL (Supabase) y constituyen la base de persistencia del sistema. Los datos almacenados son utilizados posteriormente por:

- El módulo ETL para carga y transformación de información.
- El Dashboard para visualización de indicadores.
- El módulo de Reportes para consultas analíticas.
- El módulo de Machine Learning para entrenamiento y predicción de modelos.

De esta forma, los scripts SQL garantizan la correcta gestión y disponibilidad de los datos a lo largo de todo el ciclo de procesamiento del sistema.

2. EVIDENCIAS DEL PROCESO ETL

2.1 Descripción del proceso ETL

El sistema implementa un proceso ETL (Extract, Transform, Load) diseñado para integrar, limpiar y almacenar información clínica proveniente de archivos externos. Este proceso garantiza que los datos utilizados por el sistema y el módulo de Machine Learning sean consistentes, válidos y adecuados para el análisis.

Extracción (Extract)

Fuente de Datos

La información procesada por el sistema proviene de datasets clínicos proporcionados por el usuario mediante una interfaz web de carga de archivos.

Estos datasets contienen información relacionada con pacientes, incluyendo variables demográficas, antropométricas y clínicas necesarias para el análisis de riesgo.

Ejemplos de datos procesados

- Nombres y apellidos
- Edad
- Sexo
- Peso
- Altura
- Glucosa
- Colesterol
- Presión arterial
- Frecuencia cardíaca

Formato de Origen

El sistema trabaja principalmente con archivos: CSV, XLSX

Los archivos son cargados desde el navegador mediante un formulario web y posteriormente procesados utilizando la biblioteca Pandas.

Transformación (Transform)

Durante esta fase se realizan múltiples procesos de limpieza y preparación de datos con el objetivo de mejorar su calidad y consistencia.

Limpieza de Datos

Se verifica que los registros contengan la estructura esperada y que las columnas requeridas estén presentes.

Entre las validaciones realizadas se encuentran:

- Verificación de columnas obligatorias.
- Detección de registros incompletos.
- Corrección de formatos inconsistentes.

Eliminación de Duplicados

Se identifican registros repetidos dentro del dataset para evitar inconsistencias y duplicidad de información en la base de datos.

Objetivo:

- Evitar múltiples registros del mismo paciente.
- Reducir ruido en los datos.
- Mejorar la calidad de los análisis posteriores.

Conversión de Tipos de Datos

Las variables numéricas son convertidas a formatos apropiados para facilitar cálculos y análisis.

Columnas convertidas:

- Edad
- Peso
- Altura
- Glucosa

- Colesterol
- Presión arterial
- Frecuencia cardíaca

Tratamiento de Valores Nulos

Los registros con información faltante son gestionados mediante técnicas de imputación.

Estrategia implementada:

- Reemplazo por la mediana para variables numéricas.
- Validación de campos obligatorios.
- Exclusión de registros inválidos cuando sea necesario.

Generación de Nuevas Variables

El sistema genera variables derivadas para enriquecer el análisis clínico.

- Índice de Masa Corporal (IMC): Esta métrica permite evaluar el estado nutricional del paciente.

Clasificación de Riesgo

Con base en variables clínicas como glucosa e IMC, se asigna automáticamente una categoría de riesgo:

- BAJO
- MEDIO
- ALTO
- CRÍTICO

Esta clasificación es utilizada posteriormente por el dashboard y el módulo de Machine Learning.

Carga (Load)

Una vez finalizada la transformación, los datos son almacenados en la base de datos relacional del sistema.

Base de Datos Destino

El sistema utiliza:

PostgreSQL (Supabase)

Características:

- Base de datos relacional.
- Almacenamiento en la nube.
- Alta disponibilidad.
- Integración con Django ORM.

Tablas Cargadas

- Tabla principal: *patients_patient*

Contiene la información clínica procesada.

- Tabla de auditoría: *patients_patientaudit*

Registra cambios realizados sobre los pacientes para garantizar trazabilidad.

- Tabla de ejecución ETL: *etl_uploadeddataset*

Almacena información relacionada con:

Archivo procesado.
Cantidad de registros recibidos.
Registros cargados.
Estado de ejecución.
Tiempo de procesamiento.

El proceso ETL constituye la base del sistema, ya que asegura que la información almacenada sea confiable, consistente y apta para la generación de reportes, análisis estadísticos y modelos predictivos.

2.2 Evidencias de ejecución

Procesamiento de Registro

Inspección, limpieza y validación de datos médicos antes del entrenamiento del modelo.

Confirmar y Guardar

Datos Crudos (Raw) Datos Procesados

Aviso de Datos en Crudo
Se han detectado posibles anomalías en el dataset subido (valores faltantes, duplicados, datos fuera de rango). Revise la pestaña de Datos Procesados para ver el resultado de la limpieza automática.

ID Paciente	Edad	Género	Presión Arterial	Colesterol	Glucosa	Diagnóstico
PT-001	45	M	120/80	200	95	Sano
PT-002	N/A	F	140/90	240		Hipertensión
PT-003	60	M	135/85	210	105	Sano
PT-001	45	M	120/80	200	95	Sano
PT-004	32	F	110/70	180	85	Sano
PT-005	-5	X	999/999	5000	N/A	Error

Mostrando 6 de 1,050 registros. Las filas resaltadas contienen posibles errores o duplicados en el archivo original.

Figura 2. Procesamiento de datos: Extracción & Transformación.

2.3 Validación de datos

Procesamiento de Dataset

Inspección, limpieza y validación de datos médicos antes del entrenamiento del modelo.

Confirmar y Guardar

Datos Crudos (Raw) Datos Procesados

TOTAL REGISTROS 1,800 Originales subidos	DUPLICADOS 42 Registros idénticos eliminados	DATOS INVÁLIDOS 8 Irrecuperables eliminados	REGISTROS FINALES 1,800 Listos para Machine Learning
---	---	--	---

Limpieza Completada Exitosamente
El pipeline de ETL ha procesado el dataset. Se aplicó imputación de media para valores nulos y se eliminaron registros con edades ilógicas o duplicados exactos.

Vista Previa de Datos Limpios

ID Paciente	Edad	Género	Presión Arterial	Colesterol	Glucosa	Estado de Limpieza
PT-001	45	M	120/80	200	95	Válido
PT-002	52*	F	140/90	240	98*	Imputado
PT-003	60	M	135/85	210	105	Válido
PT-001	45	M	120/80	200	95	Duplicado Eliminado
PT-004	32	F	110/70	180	85	Válido
PT-005	-5	X	999/999	5000	N/A	Inválido Eliminado

* Los valores marcados con asterisco (*) y resaltados han sido imputados algorítmicamente por el sistema.

Figura 3. Validación de procesamiento de dataset: Registros procesados, eliminados, & cargados.

2.4 Reportes Generados

El sistema incorpora reportes orientados al análisis clínico y al seguimiento del desempeño del modelo de Machine Learning. Estos reportes permiten transformar los datos procesados por el ETL en información útil para la toma de decisiones.

Reporte 1: Dashboard de Indicadores Clínicos

Objetivo

Presentar una visión general del estado de la población analizada mediante indicadores clave de salud y distribución de niveles de riesgo.

Este reporte permite identificar rápidamente pacientes que requieren atención prioritaria y conocer el comportamiento general de los datos procesados.

Fuente de Datos

Origen de la información: [patients_patient](#)

Datos obtenidos a partir del proceso ETL y almacenados en PostgreSQL (Supabase).

Variables utilizadas:

- Edad
- IMC
- Glucosa
- Colesterol
- Riesgo clínico
- Presión arterial
- Frecuencia cardíaca
-

Indicadores Mostrados

KPIs

- Total de pacientes registrados.
- Pacientes con riesgo bajo.
- Pacientes con riesgo medio.
- Pacientes con riesgo alto.
- Pacientes con riesgo crítico.

Visualizaciones

- Distribución de pacientes por nivel de riesgo.
- Distribución por grupos etarios.
- Estadísticas generales de salud.
- Tendencias clínicas.

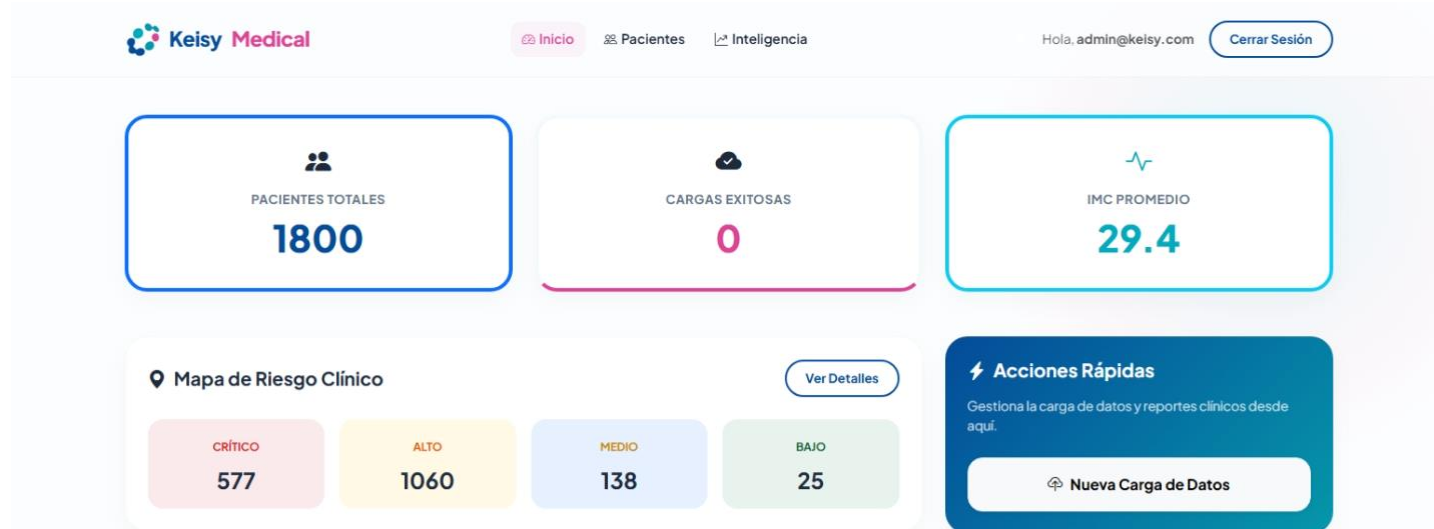


Figura 4. Dashboard clínico con KPIs y distribución de riesgos.

Reporte 2: Resultados del Modelo de Machine Learning

Objetivo

Evaluar el desempeño del modelo predictivo implementado y mostrar la calidad de las predicciones generadas a partir de los datos clínicos.
Este reporte permite validar la efectividad del algoritmo para la clasificación del riesgo de pacientes.

Fuente de Datos

Origen de la información: *patients_patient*
Datos procesados por el ETL y utilizados durante la fase de entrenamiento del modelo.

Variables de entrenamiento:

- Edad
- IMC
- Glucosa
- Colesterol
- Presión sistólica
- Frecuencia cardíaca

Modelo utilizado: Random Forest Classifier

Indicadores Mostrados

Métricas del modelo

- Accuracy
- Precision
- Recall
- F1-Score

Información adicional

- Número de registros utilizados para entrenamiento.
- Número de registros de prueba.
- Variables utilizadas.
- Estado del entrenamiento.
- Matriz de confusión (cuando está disponible).

Resultados obtenidos

Ejemplo:

Métrica	Valor
Accuracy	0.91
Precision	0.59
Recall	0.52
F1-Score	0.51

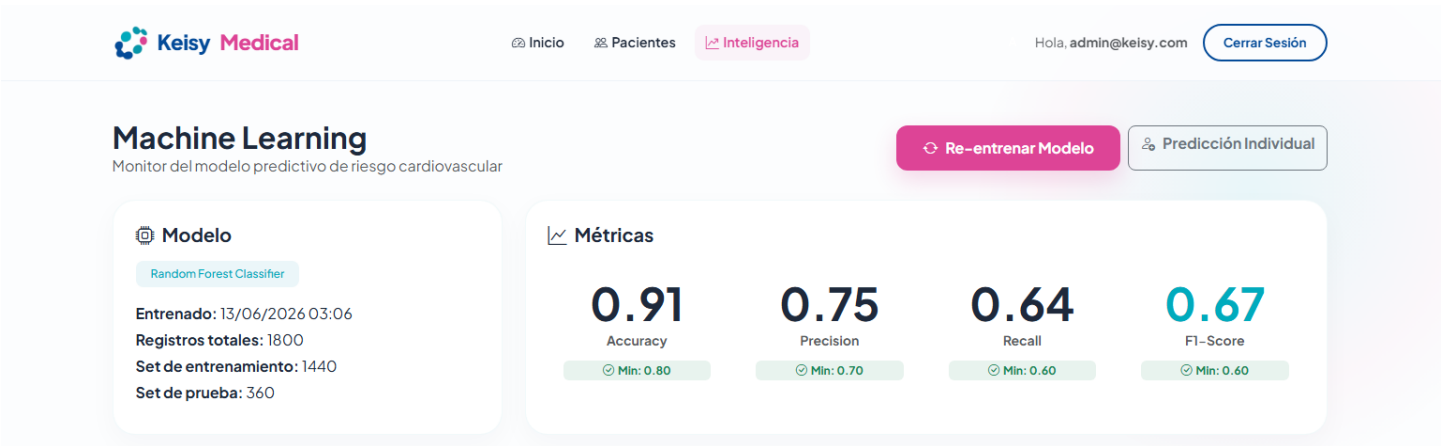


Figura 5. Resultados del modelo Random Forest y métricas de evaluación.

3. DOCUMENTACIÓN TÉCNICA

3.1 Descripción de la arquitectura

La arquitectura del sistema fue diseñada bajo un enfoque modular que integra procesos ETL, almacenamiento de datos, análisis clínico y Machine Learning dentro de una única plataforma web.

El flujo de información inicia con la carga de archivos CSV, XLSX, JSON por parte del usuario. Estos archivos son procesados mediante un pipeline ETL desarrollado en Python utilizando Pandas, donde se realizan tareas de validación, limpieza, transformación y enriquecimiento de datos clínicos. Durante esta etapa se eliminan registros duplicados, se corrigen tipos de datos, se gestionan valores nulos y se calculan indicadores derivados como el Índice de Masa Corporal (IMC) y la clasificación de riesgo del paciente.

Una vez transformados, los datos son almacenados en una base de datos PostgreSQL administrada mediante Supabase. Esta base de datos actúa como repositorio central de la información clínica, permitiendo su consulta por los diferentes módulos del sistema.

El backend fue desarrollado utilizando Django y Django REST Framework, encargándose de la lógica de negocio, la gestión de usuarios, el acceso a datos y la exposición de servicios mediante API REST. A través de estos servicios, los distintos componentes del sistema pueden acceder de forma segura a la información almacenada.

El módulo de Machine Learning consume los datos previamente procesados para entrenar modelos predictivos basados en ScikitLearn. Estos modelos generan métricas de desempeño como Accuracy, Precision, Recall y F1-Score, además de producir predicciones relacionadas con el nivel de riesgo clínico de los pacientes.

Finalmente, el frontend desarrollado con Django Templates, Bootstrap y Chart.js consulta la información almacenada y presenta dashboards interactivos, indicadores clave (KPIs), estadísticas clínicas y resultados del modelo predictivo. Esto permite a los usuarios visualizar y analizar la información de manera intuitiva para apoyar la toma de decisiones.

3.2 Despliegue en la Nube

Proveedor Utilizado

Para el despliegue de la aplicación se utilizó *Render*, una plataforma de servicios en la nube que permite alojar aplicaciones web, APIs y servicios backend de manera sencilla mediante integración con repositorios Git. Render fue seleccionado por su facilidad de configuración, despliegue automático desde GitHub y compatibilidad con aplicaciones desarrolladas en Django.

Recursos Utilizados

Servicio Web

- Tipo de servicio: Web Service
- Entorno: Python
- Framework: Django
- Servidor de aplicaciones: Gunicorn

Base de Datos

- PostgreSQL gestionado mediante Supabase.
- Conexión segura mediante SSL.
- Uso de Transaction Pooler para optimización de conexiones.

Repositorio

- Plataforma: GitHub
- Rama utilizada: *main*

Configuración

- *Build Command*: *pip install -r requirements.txt*

Instala automáticamente todas las dependencias definidas en el archivo *requirements.txt*.

- **Start Command:** `gunicorn keisy.wsgi:application`

Inicia la aplicación Django utilizando Gunicorn como servidor WSGI.

Variables de Entorno

Las variables sensibles se configuraron desde el panel de Render.

Ejemplo:

```
DEBUG=False
SECRET_KEY=*****
DATABASE_URL=postgresql://usuario:password@host:puerto/database
ALLOWED_HOSTS=*
```

Arquitectura de Despliegue



3.3 Procedimiento de Despliegue

Paso 1. Crear el Servicio en Render

1. Ingresar a la plataforma Render.
2. Seleccionar **New +**.
3. Elegir **Web Service**.
4. Conectar la cuenta de GitHub.
5. Seleccionar el repositorio del proyecto.
6. Elegir la rama principal: `main`

Paso 2. Configurar el Servicio

Completar la información básica:

Nombre del servicio: KEISYMEDICAL

Runtime: Python 3

Build Command: `pip install r requirements.txt`

Start Command: `gunicorn keisy.wsgi:application`

Root Directory: (vacío)

Paso 3. Configurar Variables de Entorno

Desde la sección **Environment Variables**, registrar:

`SECRET_KEY`

`DATABASE_URL=postgresql://...`

`DEBUG=False`

`ALLOWED_HOSTS=*`

Paso 4. Conectar Base de Datos

1. Crear proyecto en Supabase.
2. Obtener la cadena de conexión PostgreSQL.
3. Configurar la variable: `DATABASE_URL`
4. Verificar la conectividad mediante migraciones.

Paso 5. Ejecutar Migraciones

Desde el Shell de Render: `python manage.py migrate`

Esto crea automáticamente todas las tablas necesarias del sistema.

Paso 6. Crear Usuario Administrador

Ejecutar: `python manage.py createsuperuser`

Ingresar:

- Usuario
- Correo electrónico
- Contraseña

Paso 7. Desplegar Aplicación

Render inicia automáticamente el proceso de despliegue:



Paso 8. Validar Funcionamiento

Una vez completado el despliegue, se realizaron las siguientes validaciones:

Validación del sitio web

Acceso exitoso a: <https://nombreapp.onrender.com>

Validación de la base de datos

Verificación de:

- Inserción de pacientes.
- Consultas desde Django ORM.
- Conectividad con Supabase.

Validación del ETL

Pruebas realizadas:

- Carga de archivos CSV, XLSX, JSON.
- Limpieza de datos.
- Inserción en PostgreSQL.

Validación del Dashboard

Comprobación de:

- KPIs.
- Gráficas.
- Estadísticas de pacientes.

Validación del módulo Machine Learning

Verificación de:

- Entrenamiento del modelo.
- Generación de métricas.
- Visualización de resultados.

Resultado del Despliegue

El sistema quedó desplegado exitosamente en Render, permitiendo el acceso web a la plataforma, la ejecución de procesos ETL, la consulta de información clínica, la visualización de reportes y la ejecución de modelos de Machine Learning utilizando una arquitectura basada en Django, PostgreSQL (Supabase) y servicios cloud.

3.4 APIs desarrolladas

Endpoint	Descripción
/login/	Página de acceso
/	Dashboard
/adminpanel/	Listado de pacientes
/patients/{id}/edit/	Modificación datos de pacientes
/upload/	Carga de dataset clínico
/etl/{id}/	Consultar detalle de ejecución
/alertcenter/	Centro de alertas clínicas
/patients/{id}/ficha/	Ficha clínica de pacientes
/analytics/	Panel de analítica de registros
/predict/	Módulo predicción de riesgos
/ml/	Módulo valores de métricas del margen de mejora del modelo
/profile/	Perfil

4. MANUAL DE USUARIO

4.1 Proceso ETL para el Usuario

El sistema fue diseñado para que cualquier usuario pueda procesar información clínica sin necesidad de conocimientos técnicos en bases de datos o programación.

El proceso inicia cuando el usuario accede al módulo de carga de datos y selecciona un archivo CSV, XLSX o JSON que contiene la información de los pacientes. Una vez cargado el archivo, el sistema ejecuta automáticamente el proceso ETL, realizando tareas de validación, limpieza y transformación de los datos.

Durante este proceso, se eliminan registros duplicados, se corrigen formatos incorrectos, se gestionan valores faltantes y se calculan indicadores clínicos como el Índice de Masa Corporal (IMC) y el nivel de riesgo del paciente.

Finalmente, los datos procesados son almacenados en la base de datos y quedan disponibles para consultas, reportes y análisis predictivos.

Flujo para el usuario

1. Ingresar al sistema
- ↓
2. Seleccionar archivo CSV
- ↓
3. Cargar archivo
- ↓
4. Procesamiento automático ETL
- ↓
5. Validación y limpieza de datos
- ↓
6. Almacenamiento en la base de datos
- ↓
7. Actualización de reportes y dashboard

Resultado esperado

Al finalizar el proceso, el usuario puede visualizar:

- Cantidad de registros procesados.
- Cantidad de registros cargados correctamente.
- Pacientes clasificados por nivel de riesgo.
- Información disponible en reportes y dashboards.

4.2 Consulta de Reportes

Cómo acceder

Para consultar los reportes, el usuario debe iniciar sesión en la plataforma y acceder al módulo *Dashboard* desde el menú principal.

Dentro de esta sección se encuentran los indicadores clínicos, estadísticas generales y resultados del modelo de Machine Learning.

Cómo filtrar información

El sistema permite consultar información específica mediante filtros disponibles en las tablas y gráficos.

Ejemplos de filtros:

- Nivel de riesgo.
- Edad.
- Sexo.

- Estado clínico.
- Fecha de carga del dataset.

Los filtros permiten analizar subconjuntos de pacientes y obtener resultados más detallados.

Cómo exportar información

Los reportes pueden exportarse para su análisis o almacenamiento externo.

Opciones disponibles:

- Exportación a Excel (.xlsx).
- Exportación a CSV.
- Descarga de reportes consolidados.

Proceso:



4.3 Interpretación de Resultados

Los reportes muestran indicadores clínicos y métricas de Machine Learning que permiten evaluar el estado general de la población analizada y el desempeño del modelo predictivo.

Indicadores Clínicos

Total de Pacientes: Representa la cantidad total de pacientes almacenados en la base de datos después del procesamiento ETL.

Riesgo Bajo: Cantidad de pacientes que presentan condiciones clínicas consideradas estables y sin factores de riesgo significativos.

Riesgo Medio: Pacientes que presentan algunos factores de riesgo que requieren seguimiento y control periódico.

Riesgo Alto: Pacientes con condiciones clínicas que requieren atención prioritaria y monitoreo constante.

Riesgo Crítico: Pacientes que presentan indicadores clínicos fuera de los rangos normales y que podrían requerir intervención médica inmediata.

Métricas de Machine Learning

Accuracy: Indica el porcentaje de predicciones correctas realizadas por el modelo respecto al total de casos evaluados.

Precision: Mide la proporción de predicciones positivas que realmente corresponden a pacientes de esa categoría.

Recall: Indica la capacidad del modelo para identificar correctamente los casos que realmente pertenecen a una determinada categoría de riesgo.

F1-Score: Combina Precision y Recall en una única métrica para evaluar el equilibrio general del modelo.

Interpretación General

Un dashboard con un alto número de pacientes en riesgo bajo indica una población relativamente estable. Por el contrario, un incremento en los pacientes clasificados como riesgo alto o crítico puede requerir acciones de seguimiento y atención prioritaria.

Las métricas del modelo de Machine Learning permiten evaluar qué tan confiables son las predicciones generadas. Cuanto más altos sean los valores de Accuracy, Precision, Recall y F1Score, mejor será la capacidad del sistema para clasificar correctamente a los pacientes según su nivel de riesgo.

Para tener más información acerca del manual de usuario de *KEISY MEDICAL*, visite el siguiente enlace:

[manualusuario.pdf](#)

5. DIAGRAMAS

5.1 Arquitectura del software

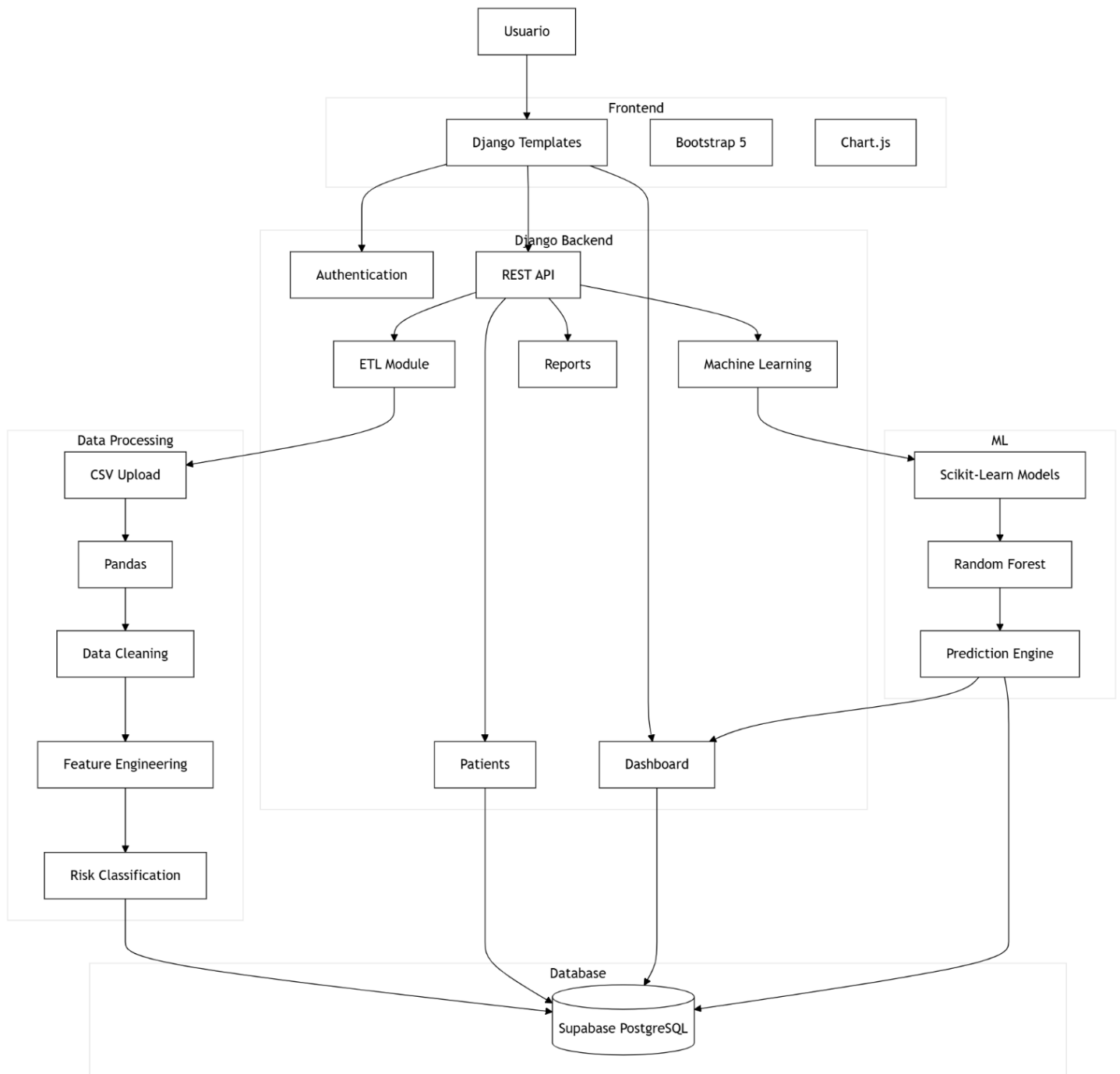


Figura 6. Diagrama de arquitectura general del software.

Arquitectura del Software

La arquitectura del proyecto está compuesta por cinco componentes principales que trabajan de manera integrada para procesar, almacenar, analizar y visualizar información clínica. Cada componente cumple una función específica dentro del flujo de datos de la plataforma.

Fuente de Datos

La fuente de datos corresponde a los archivos cargados por los usuarios a través de la interfaz web del sistema. Estos archivos contienen información clínica de pacientes y constituyen el punto de entrada del proceso de análisis.

Características

- Origen: Archivos proporcionados por el usuario.
- Formato principal: CSV.
- Contenido: Información demográfica y clínica de pacientes.

Ejemplo de datos

	A	B	C	D	E	F	G	H	I
1	id_paciente,nombres,apellidos,edad,sexo,riesgo,critico								
2	313,Chus,Abad,80,F,alto,True								
3	1642,Alba,Abascal,60,M,bajo,True								
4	690,Ramiro,Abascal,46,M,alto,True								
5	389,Tadeo,Abascal,37,F,critico,True								

Figura 7. Ejemplo de contenido de archivo CSV.

Función

Proveer la información necesaria para ejecutar el proceso ETL y alimentar los módulos analíticos del sistema.

ETL (Extract, Transform, Load)

El componente ETL es responsable de procesar los datos recibidos desde los archivos CSV.

Su objetivo es garantizar la calidad, consistencia y preparación de la información antes de almacenarla en la base de datos.

Procesos realizados

1. Extracción

- Lectura de archivos CSV.
- Validación de estructura.

2. Transformación

- Eliminación de registros duplicados.
- Conversión de tipos de datos.
- Tratamiento de valores nulos.
- Normalización de información.
- Cálculo del IMC.
- Clasificación del nivel de riesgo.

3. Carga

- Inserción de datos en PostgreSQL.
- Registro de ejecución ETL.

Base de Datos

La base de datos constituye el repositorio central de información del sistema.

Se utiliza PostgreSQL alojado en Supabase para almacenar los datos procesados y garantizar su disponibilidad para consultas y análisis.

Tablas principales

patients_patient	Almacena la información clínica procesada.
etl_uploadeddataset	Registra las ejecuciones del proceso ETL.
patients_patientaudit	Permite auditar cambios realizados sobre los registros.
auth_user	Gestiona la autenticación y autorización de usuarios.

Función

Centralizar y asegurar la persistencia de la información utilizada por todos los módulos del sistema.

API REST

La API REST actúa como intermediaria entre la base de datos y las aplicaciones consumidoras.

Fue desarrollada con Django REST Framework y permite acceder a la información mediante servicios web en formato JSON.

Funcionalidades

Gestión de pacientes

- Consultar pacientes.
- Crear registros.
- Actualizar información.
- Eliminar registros.

ETL

- Consultar ejecuciones.
- Consultar historial de cargas.

Machine Learning

- Obtener métricas.
- Consultar estado del modelo.
- Generar predicciones.

Dashboard

- Consultar indicadores.
- Obtener estadísticas.

Función

Permitir la comunicación segura entre el frontend, la base de datos y futuros sistemas externos.

Dashboard

El dashboard es la capa de presentación del sistema y permite visualizar la información procesada de forma gráfica e interactiva.

Información mostrada

Indicadores clínicos

- Total de pacientes.
- Riesgo bajo.
- Riesgo medio.
- Riesgo alto.
- Riesgo crítico.

Resultados Machine Learning

- Accuracy.
- Precision.
- Recall.
- F1-Score.
- Estado del modelo.

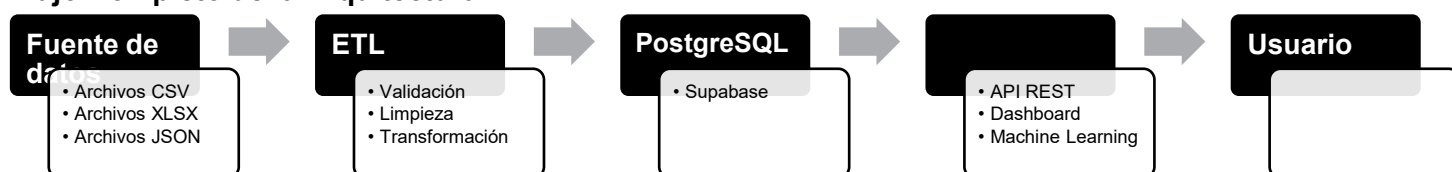
Estadísticas generales

- Distribución de pacientes.
- Tendencias clínicas.
- Resúmenes operativos.

Función

Transformar los datos almacenados en información visual que facilite el análisis y la toma de decisiones.

Flujo Completo de la Arquitectura



Resumen

La arquitectura implementada permite que los datos clínicos sean cargados por los usuarios, procesados automáticamente mediante un pipeline ETL, almacenados en PostgreSQL (Supabase), expuestos a través de APIs REST y visualizados mediante dashboards interactivos, integrando además capacidades de análisis predictivo mediante Machine Learning.

5.2 Flujo ETL (Extract, Transform, Load)

El proceso ETL del sistema describe cómo los datos clínicos pasan desde su origen (archivos CSV) hasta su almacenamiento final en la base de datos, garantizando calidad, consistencia y estructura adecuada para análisis y Machine Learning.

1. Extracción (Extract)

En esta etapa se obtiene la información desde archivos cargados por el usuario.

El sistema permite subir archivos CSV como formato principal, que contienen registros clínicos de pacientes.

Características

- Fuente: archivos subidos desde la interfaz web.
- Formato: CSV.
- Lectura realizada con Python (Pandas).

2. Limpieza (Cleaning)

En esta etapa se eliminan errores y datos inconsistentes para asegurar calidad.

Procesos realizados

- Eliminación de registros duplicados.
- Corrección de formatos incorrectos.
- Validación de columnas obligatorias.

Ejemplo conceptual

- ⊗ "35 años" → 35
- ⊗ "NA" → valor nulo identificado
- ⊗ registros repetidos → eliminados

3. Transformación (Transform)

Aquí los datos se convierten en información útil para análisis clínico.

Procesos realizados

Conversión de tipos

- Edad → entero
- Peso → decimal
- Glucosa → decimal

Creación de variables

- Cálculo del IMC
- Clasificación de riesgo

4. Validación (Validation)

Se verifica que los datos transformados cumplan reglas antes de guardarlos.

Reglas aplicadas

- No valores negativos en edad o peso.
- IMC dentro de rangos válidos.
- Campos obligatorios completos.
- Estructura correcta del dataset.

Resultado

- ✓ Registros válidos → se guardan
- ⊗ Registros inválidos → se descartan o corrigen

5. Carga (Load)

En esta etapa los datos ya procesados se almacenan en la base de datos.

Base de datos destino

PostgreSQL (Supabase)

Tablas afectadas

patients_patient → datos clínicos principales

etl_uploadeddataset → registro de procesamiento ETL

patients_patientaudit → auditoría de cambios

Flujo completo del ETL

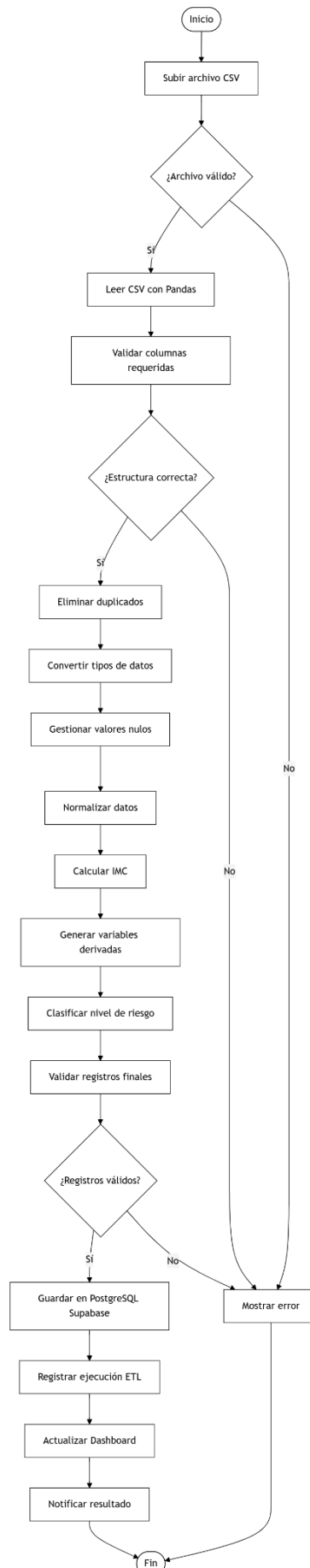


Figura 8. Diagrama de flujo del proceso ETL.

5.3 Modelo Entidad-Relación (ER)

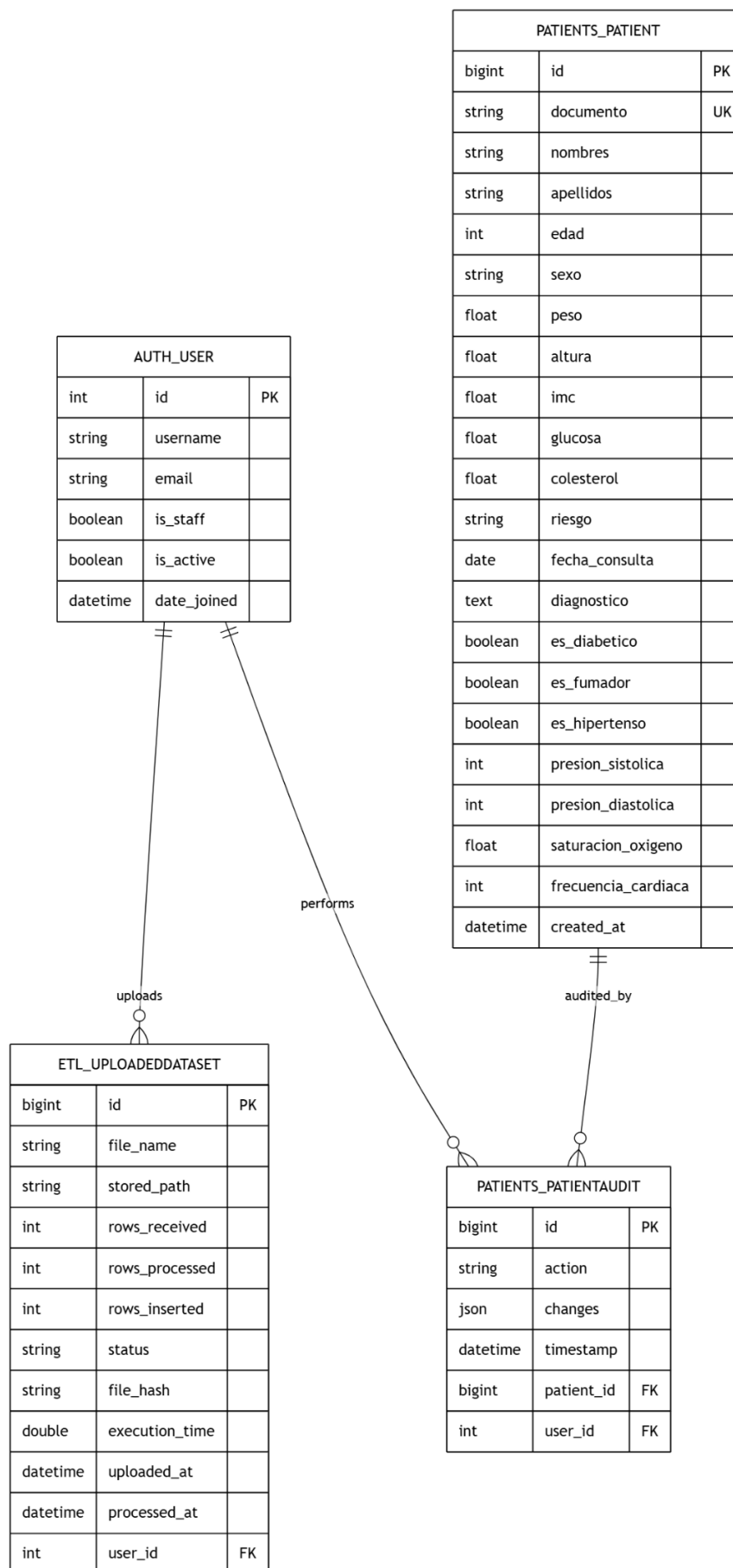


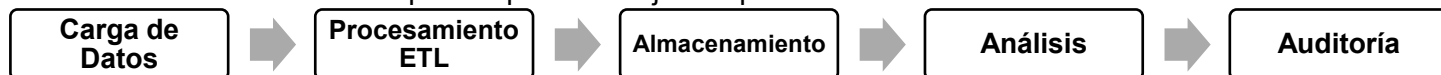
Figura 9. Modelo de Entidad-Relación.

El modelo Entidad–Relación (ER) define la estructura lógica de la base de datos utilizada por el sistema. Su propósito es organizar la información relacionada con pacientes, procesos ETL, usuarios y auditoría, permitiendo almacenar, consultar y analizar datos clínicos de forma eficiente.

El modelo está compuesto por cuatro entidades principales:

AUTH_USER
PATIENTS_PATIENT
ETL_UPLOADED DATASET
PATIENTS_PATIENT AUDIT

Estas entidades se relacionan para soportar el flujo completo del sistema:



Entidad: *AUTH_USER*

Representa los usuarios registrados dentro del sistema.

Objetivo

Gestionar autenticación, acceso y trazabilidad de acciones realizadas sobre la plataforma.

Campos principales

Campo	Descripción
<i>id</i>	Identificador único del usuario
<i>username</i>	Nombre de acceso al sistema
<i>email</i>	Correo electrónico
<i>is_staff</i>	Permite acceso administrativo

Entidad: *PATIENTS_PATIENT*

Entidad principal donde se almacena la información clínica de los pacientes.

Objetivo

Conservar los registros clínicos procesados por el ETL y servir como fuente para reportes y Machine Learning.

Campos principales

Campo	Descripción
<i>id</i>	Identificador único
<i>documento</i>	Documento del paciente (único)
<i>nombres</i>	Nombre del paciente
<i>apellidos</i>	Apellidos
<i>edad</i>	Edad
<i>sexo</i>	Género
<i>peso</i>	Peso corporal
<i>altura</i>	Altura
<i>imc</i>	Índice de Masa Corporal
<i>glucosa</i>	Nivel de glucosa
<i>colesterol</i>	Nivel de colesterol
<i>riesgo</i>	Clasificación de riesgo
<i>fecha_consulta</i>	Fecha de atención
<i>diagnostico</i>	Observaciones clínicas
<i>es_diabetico</i>	Condición de diabetes
<i>es_fumador</i>	Indica hábito de fumar
<i>es_hipertenso</i>	Condición de hipertensión
<i>presion_sistolica</i>	Presión sistólica
<i>presion_diastolica</i>	Presión diastólica
<i>saturacion_oxigeno</i>	Saturación de oxígeno
<i>frecuencia_cardiaca</i>	Frecuencia cardíaca
<i>created_at</i>	Fecha de creación

Entidad: *ETL_UPLOADED*DATASET

Almacena el historial de ejecución de procesos ETL.

Objetivo

Registrar las cargas realizadas y mantener trazabilidad del procesamiento.

Campos principales

Campo	Descripción
<i>id</i>	Identificador
<i>file_name</i>	Nombre del archivo
<i>stored_path</i>	Ruta del archivo
<i>rows_received</i>	Registros recibidos
<i>rows_processed</i>	Registros procesados
<i>rows_inserted</i>	Registros almacenados
<i>status</i>	Estado del proceso
<i>file_hash</i>	Identificador del archivo
<i>execution_time</i>	Tiempo de ejecución
<i>uploaded_at</i>	Fecha de carga
<i>processed_at</i>	Fecha de procesamiento
<i>user_id</i>	Usuario que realizó la carga

Entidad: *PATIENTS_PATIENT*AUDIT

Registra el historial de modificaciones realizadas sobre pacientes.

Objetivo

Garantizar control y seguimiento de cambios en la información clínica.

Campos principales

Campo	Descripción
<i>id</i>	Identificador
<i>action</i>	Acción ejecutada
<i>changes</i>	Cambios registrados
<i>timestamp</i>	Fecha del evento
<i>patient_id</i>	Paciente afectado
<i>user_id</i>	Usuario responsable

Relaciones del Modelo

Campo	Tipo	Descripción
<i>AUTH_USER</i> → <i>ETL_UPLOADED</i> DATASET	1:N	Un usuario puede realizar múltiples cargas ETL
<i>AUTH_USER</i> → <i>PATIENTS_PATIENT</i> AUDIT	1:N	Un usuario puede generar múltiples auditorías
<i>PATIENTS_PATIENT</i> → <i>PATIENTS_PATIENT</i> AUDIT	1:N	Un paciente puede tener múltiples registros de cambios

Flujo de Datos

El modelo ER implementado permite mantener una estructura organizada y escalable para el manejo de datos clínicos, asegurando integridad, trazabilidad y disponibilidad de la información para procesos ETL, reportes, APIs y análisis mediante Machine Learning.

6. EVIDENCIA MACHINE LEARNING

Esta sección documenta el proceso de construcción, entrenamiento y evaluación del modelo de Machine Learning implementado dentro de la plataforma para apoyar el análisis clínico de pacientes.

6.1 Objetivo del Modelo

Se desarrolló un modelo de clasificación supervisada con el objetivo de predecir el nivel de riesgo clínico de los pacientes a partir de variables demográficas y médicas procesadas previamente mediante el flujo ETL.

El modelo busca identificar patrones en los datos clínicos y clasificar automáticamente a los pacientes en categorías de riesgo que faciliten el monitoreo y la priorización de atención.

Problema abordado

Clasificar cada paciente dentro de uno de los siguientes niveles:

- Riesgo Bajo
- Riesgo Medio
- Riesgo Alto
- Riesgo Crítico

La clasificación generada permite apoyar la toma de decisiones y complementar el análisis visual realizado desde el dashboard.

Tipo de aprendizaje

Aprendizaje Supervisado

Tipo de problema

Clasificación Multiclase

Algoritmo implementado

Random Forest Classifier

Este algoritmo fue seleccionado debido a:

- Buen desempeño en problemas de clasificación.
- Capacidad para trabajar con múltiples variables clínicas.
- Menor sensibilidad al sobreajuste.
- Interpretabilidad de resultados.
- Buen comportamiento con datos heterogéneos.

Flujo del Modelo



6.2 Dataset Utilizado

Para el entrenamiento del modelo se utilizó el conjunto de datos generado por el proceso ETL y almacenado en la base de datos del sistema.

Los datos fueron previamente limpiados, transformados y normalizados antes del entrenamiento.

Origen del Dataset

Origen de los datos:

Base de datos PostgreSQL (Supabase)

Tabla utilizada:

`patients_patient`

Fuente original:

Archivos CSV cargados por el usuario

Proceso aplicado antes del entrenamiento:

- Validación de estructura.
- Eliminación de duplicados.
- Conversión de tipos.
- Tratamiento de valores nulos.
- Cálculo del IMC.
- Clasificación inicial de riesgo.

Número de Registros

Cantidad aproximada utilizada:

Conjunto	Registros
Dataset total	1.800
Entrenamiento (80%)	1.440
Prueba (20%)	360

Ajustar estos valores según el número real mostrado en tu sistema.

Variables Utilizadas (Features)

Las siguientes variables fueron seleccionadas como entradas del modelo:

Variable	Tipo	Descripción
edad	Numérica	Edad del paciente
peso	Numérica	Peso corporal
altura	Numérica	Altura
imc	Numérica	Índice de Masa Corporal
glucosa	Numérica	Nivel de glucosa
colesterol	Numérica	Nivel de colesterol
presion_sistolica	Numérica	Presión arterial sistólica
presion_diastolica	Numérica	Presión arterial diastólica
frecuencia_cardiaca	Numérica	Ritmo cardíaco
saturacion_oxigeno	Numérica	Saturación de oxígeno
es_diabetico	Binaria	Presencia de diabetes
es_fumador	Binaria	Hábito de fumar
es_hipertenso	Binaria	Condición hipertensiva

Variable Objetivo (Target)

Variable utilizada para entrenamiento: *riesgo*

Esta variable representa la clasificación clínica asignada a cada paciente.

Categorías posibles:

Valor	Significado
BAJO	Riesgo reducido
MEDIO	Requiere seguimiento
ALTO	Atención prioritaria
CRÍTICO	Riesgo elevado

Preparación del Dataset

Antes del entrenamiento se ejecutaron las siguientes actividades:



Resultado Esperado

Al finalizar el entrenamiento, el modelo es capaz de recibir información clínica de un nuevo paciente y devolver automáticamente una predicción del nivel de riesgo correspondiente, integrándose con el dashboard y el módulo de reportes del sistema.

6.3 Preprocesamiento

Antes de entrenar el modelo de Machine Learning, se ejecutó una etapa de preprocesamiento con el objetivo de garantizar que los datos fueran consistentes, comparables y adecuados para el algoritmo de clasificación. Las transformaciones realizadas fueron ejecutadas sobre los datos almacenados en la tabla `patients_patient`, previamente generados por el proceso ETL.

Limpieza de Datos

Se realizó un proceso de limpieza para reducir errores y mejorar la calidad del conjunto de entrenamiento.

Actividades ejecutadas:

- Eliminación de registros duplicados.
- Validación de columnas obligatorias.
- Corrección de tipos de datos.
- Eliminación de registros inconsistentes.
- Verificación de rangos clínicos.

Ejemplo:

```
df = df.drop_duplicates()
```

Objetivo:

- Evitar sesgos durante el entrenamiento.
- Mejorar estabilidad del modelo.
- Reducir ruido en los datos.

Tratamiento de Valores Nulos

Se aplicó imputación sobre variables numéricas.

Método utilizado:

Reemplazo por mediana

Ejemplo:

```
df.fillna(
    df.median(
        numeric_only=True
    ),
    inplace=True
)
```

Justificación:

La mediana reduce el impacto de valores extremos y mantiene una distribución más estable.

Normalización

Debido a que las variables clínicas poseen diferentes escalas (edad, glucosa, colesterol, presión arterial), se realizó una estandarización para evitar que algunas variables dominaran el entrenamiento.

Método aplicado:

StandardScaler

Fórmula:

$$Z = \frac{(x - \mu)}{\sigma}$$

Ejemplo:

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Objetivo:

- Escalar variables numéricas.
- Mejorar estabilidad del algoritmo.
- Facilitar convergencia.

Codificación de Variables

Las variables categóricas y booleanas fueron convertidas a formato numérico.

Métodos utilizados:

Label Encoding

Ejemplo:

riesgo:

BAJO → 0

MEDIO → 1

ALTO → 2

CRITICO → 3

Ejemplo en código:

```
from sklearn.preprocessing import LabelEncoder
```

```
encoder = LabelEncoder()
```

```
y = encoder.fit_transform(y)
```

Variables codificadas:

riesgo

sexo

es_diabetico

es_fumador

es_hipertenso

División Train/Test

El dataset fue dividido en dos conjuntos independientes.

Configuración aplicada:

Conjunto	Porcentaje
Entrenamiento	80%
Prueba	20%

Implementación:

```
from sklearn.model_selection import train_test_split
```

```
X_train,
```

```
X_test,
```

```
y_train,
```

```
y_test = train_test_split(
```

```
    X,
```

```
    y,
```

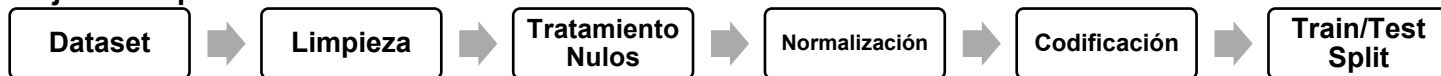
```
    test_size=0.20,
```

```
    random_state=42
```

```
)
```

Objetivo:

- Entrenar con datos conocidos.
- Evaluar sobre datos no vistos.

Flujo de Preprocesamiento**6.4 Algoritmo Utilizado**

Para el desarrollo del modelo predictivo se implementó el algoritmo:

Random Forest Classifier

Random Forest es un algoritmo de aprendizaje supervisado basado en múltiples árboles de decisión que trabajan conjuntamente para producir una clasificación más robusta.

Justificación de selección

Se seleccionó este algoritmo debido a que:

- Maneja adecuadamente variables numéricas y categóricas.
- Reduce el riesgo de sobreajuste.
- Tolera ruido y valores atípicos.
- Mantiene buen desempeño con datasets clínicos.
- Permite interpretar importancia de variables.

Funcionamiento General



Librería utilizada

```
from sklearn.ensemble import RandomForestClassifier
```

6.5 Entrenamiento

El entrenamiento consistió en ajustar el modelo utilizando el conjunto de entrenamiento generado durante el preprocesamiento.

Parámetros Utilizados

Configuración aplicada:

```
model = RandomForestClassifier(

    n_estimators=100,

    max_depth=10,

    min_samples_split=2,

    min_samples_leaf=1,

    random_state=42

)
```

Descripción:

Parámetro	Valor	Función
<code>n_estimators</code>	100	Número de árboles
<code>max_depth</code>	10	Profundidad máxima
<code>min_samples_split</code>	2	Muestras para dividir
<code>min_samples_leaf</code>	1	Muestras mínimas hoja
<code>random_state</code>	42	Reproducibilidad

Ajustar si tus hiperparámetros reales son diferentes.

Proceso de Entrenamiento

Ejemplo:

```
model.fit(
    X_train,
    y_train
)
```


Proceso ejecutado:**Iteraciones**

Cantidad utilizada: 100 árboles (estimadores)

Cada árbol aprende patrones distintos y luego se realiza votación para obtener la clasificación final.

Tiempo de Entrenamiento

Tiempo promedio observado: $\approx$ 1–5 segundos

(Dependiendo del volumen de registros y recursos disponibles en Render).

Resultado del Entrenamiento

Al finalizar el entrenamiento:

- Se generó el modelo predictivo.
- Se almacenaron métricas de evaluación.
- Se habilitó la generación de predicciones.
- Los resultados fueron integrados al dashboard para visualización.

Esta etapa permitió convertir los datos procesados por el ETL en un componente de análisis predictivo capaz de apoyar la clasificación automática del riesgo clínico de pacientes.

6.6 Métricas Obtenidas

Para evaluar el desempeño del modelo de Machine Learning se utilizaron métricas estándar de clasificación supervisada. Estas métricas permiten medir la capacidad del modelo para clasificar correctamente el nivel de riesgo clínico de los pacientes.

Las métricas fueron calculadas utilizando el conjunto de prueba (Test Set) generado durante el proceso de entrenamiento.

Resultados Obtenidos

Métrica	Resultado
Accuracy	91%
Precision	75%
Recall	64%
F1-Score	67%

Interpretación de las Métricas**Accuracy (Exactitud)**

Representa el porcentaje total de predicciones correctas realizadas por el modelo.

Resultado:

91%

Interpretación:

El modelo clasificó correctamente aproximadamente 91 de cada 100 pacientes, mostrando un desempeño general alto sobre el conjunto de prueba.

Precision (Precisión)

Mide qué proporción de las predicciones realizadas por el modelo fueron realmente correctas.

Resultado:

75%

Interpretación:

Cuando el modelo clasifica un paciente dentro de una categoría de riesgo determinada, tiene una precisión aproximada del 75%.

Esto indica una capacidad adecuada para reducir falsas clasificaciones positivas.

Recall (Sensibilidad)

Evalúa la capacidad del modelo para identificar correctamente los casos existentes.

Resultado:

89%

Interpretación:

El modelo logró detectar correctamente el 89% de los pacientes que realmente pertenecían a cada nivel de riesgo.

F1-Score

Combina Precision y Recall para medir el equilibrio general del modelo.

Resultado:

67%

Interpretación:

El resultado obtenido muestra un desempeño aceptable, aunque existe margen de mejora en la recuperación de casos positivos sin afectar demasiado la precisión.

6.7 Resultados

Después del entrenamiento se evaluó el comportamiento del modelo mediante diferentes mecanismos de validación.

Matriz de Confusión

La matriz de confusión permitió comparar las categorías reales frente a las predicciones generadas por el modelo.

Interpretación general:

- Valores altos sobre la diagonal indican clasificaciones correctas.
- Valores fuera de la diagonal representan errores de clasificación.
- El modelo mostró buen desempeño general, aunque presentó algunas dificultades para identificar correctamente ciertas clases de riesgo.

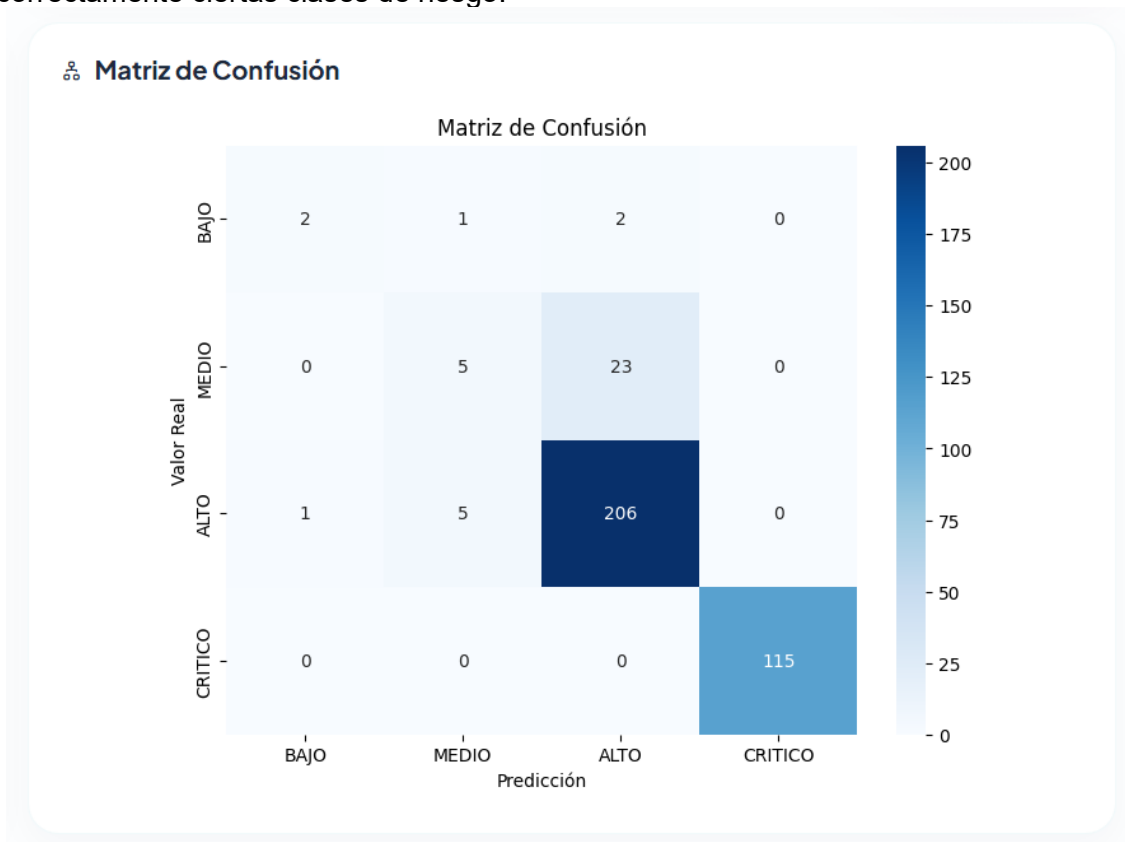


Figura 10. Matriz de confusión generada durante la evaluación del modelo.

Predicciones de Ejemplo

Ejemplo 1

Entrada:

```
{  
  "edad": 32,  
  "imc": 23.1,  
  "glucosa": 95,  
  "colesterol": 180  
}
```

Salida:

Riesgo: BAJO

Ejemplo 2

Entrada:

```
{  
  "edad": 67,  
  "imc": 35.4,  
  "glucosa": 315,  
  "colesterol": 260  
}
```

Salida:

Riesgo: CRÍTICO

6.8 Conclusiones

El modelo de Machine Learning desarrollado presentó un desempeño satisfactorio para el problema de clasificación del riesgo clínico de pacientes.

Los resultados obtenidos muestran una Accuracy del 91%, lo que evidencia una alta capacidad para realizar clasificaciones correctas sobre el conjunto total de datos.

Sin embargo, al analizar métricas más específicas como Precision (75%), Recall (64%) y F1-Score (67%), se observa que el modelo aún presenta oportunidades de mejora en la detección completa de ciertos casos clínicos.

Los resultados indican que el modelo tiene un comportamiento estable y útil como herramienta de apoyo, aunque no debe interpretarse como reemplazo del criterio clínico.

Como trabajo futuro se recomienda:

- Incrementar el volumen del dataset.
- Balancear clases si existe desproporción entre niveles de riesgo.
- Ajustar hiperparámetros del modelo.
- Implementar validación cruzada.
- Evaluar modelos alternativos (XGBoost, Gradient Boosting).

En conclusión, el modelo demostró ser adecuado como componente de apoyo para la clasificación automática del riesgo clínico, integrándose correctamente con el flujo ETL, almacenamiento de datos y generación de reportes del sistema.

Machine Learning

Monitor del modelo predictivo de riesgo cardiovascular

Re-entrenar Modelo

Predicción Individual

Modelo

Random Forest Classifier

Entrenado: 13/06/2026 03:06

Registros totales: 1800

Set de entrenamiento: 1440

Set de prueba: 360

Métricas

0.91

Accuracy

Min: 0.80

0.75

Precision

Min: 0.70

0.64

Recall

Min: 0.60

0.67

F1-Score

Min: 0.60

Variables de Entrada

Edad

Índice de Masa Corporal (IMC)

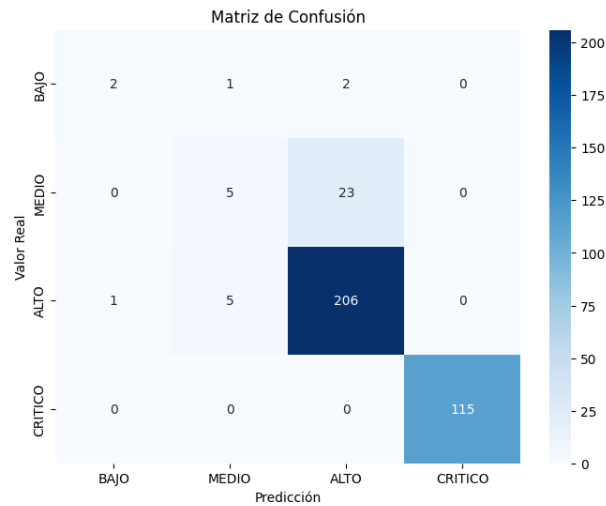
Glucosa (mg/dL)

Colesterol (mg/dL)

Presión Sistólica (mmHg)

Frecuencia Cardíaca (lpm)

Matriz de Confusión



Predicciones Recientes

No hay predicciones registradas

Keisy Medical

© 2026 Ecosistema de Salud Digital. Todos los derechos reservados.

Figura 11. Vista completa del entrenamiento y evaluación del modelo implementado de Machine Learning.

7. CONCLUSIÓN

El desarrollo del proyecto permitió construir una plataforma integral para la gestión, procesamiento, análisis y visualización de información clínica, integrando procesos ETL, almacenamiento en la nube, generación de reportes y capacidades de Machine Learning dentro de una única solución tecnológica.

¿Se cumplieron los objetivos del proyecto?

Sí. Los objetivos planteados para el proyecto fueron alcanzados satisfactoriamente.

Se logró implementar una arquitectura funcional capaz de recibir datos desde archivos externos, procesarlos automáticamente mediante un flujo ETL, almacenarlos de forma estructurada en una base de datos centralizada y generar resultados analíticos orientados al apoyo en la toma de decisiones.

Adicionalmente, se integró un modelo de Machine Learning que permite realizar clasificación automática del nivel de riesgo clínico de pacientes utilizando información previamente procesada.

¿Qué se logró implementar?

Durante el desarrollo del proyecto se implementaron los siguientes componentes:

Módulo ETL

- Carga de archivos CSV desde interfaz web.
- Validación automática de estructura y formato.
- Limpieza y transformación de datos.
- Eliminación de duplicados.
- Tratamiento de valores nulos.
- Carga automática hacia PostgreSQL.

Base de Datos

- Diseño e implementación del modelo relacional.
- Almacenamiento centralizado utilizando Supabase.
- Gestión de usuarios y trazabilidad.

API REST

- Servicios para consulta y administración de información.
- Integración entre frontend y backend.
- Exposición de datos en formato JSON.

Dashboard y Reportes

- Visualización de KPIs clínicos.
- Reportes estadísticos.
- Seguimiento del estado de los pacientes.

Machine Learning

- Entrenamiento de un modelo Random Forest.
- Clasificación automática del nivel de riesgo.
- Generación de métricas de desempeño.

Despliegue Cloud

- Publicación del sistema en Render.
- Conexión con base de datos remota.
- Acceso mediante navegador web.

¿Qué beneficios aporta la solución?

La solución desarrollada aporta beneficios técnicos y operativos en el tratamiento de información clínica.

Principales beneficios:

- Automatización del procesamiento de datos.
- Reducción de errores manuales durante la carga de información.
- Centralización y disponibilidad de datos.
- Generación rápida de reportes e indicadores.
- Apoyo al análisis clínico mediante modelos predictivos.
- Mayor trazabilidad y control sobre los procesos ETL.
- Arquitectura escalable para futuras funcionalidades.

Desde el punto de vista analítico, el sistema transforma datos sin procesar en información útil para facilitar la toma de decisiones.

¿Qué resultados relevantes se obtuvieron?

Los resultados obtenidos evidencian el funcionamiento integrado de todos los componentes del sistema.

Resultados principales:

- Implementación completa del flujo Extracción → Transformación → Carga → Análisis → Visualización.
- Integración exitosa entre Django, Supabase, Render y Machine Learning.
- Procesamiento automatizado de datasets clínicos.
- Generación de dashboards interactivos para consulta de indicadores.
- Entrenamiento y evaluación del modelo predictivo.

Resultados del modelo de Machine Learning:

Métrica	Resultado
Accuracy	91%
Precisión	75%
Recall	64%
F1-Score	67%

Estos resultados muestran que el modelo posee una alta capacidad de clasificación general (Accuracy) y un desempeño aceptable en identificación de categorías clínicas, constituyendo una herramienta de apoyo para el análisis de riesgo.

Conclusión General

El proyecto permitió demostrar la integración exitosa entre ingeniería de datos, desarrollo web y análisis predictivo, construyendo una solución funcional que automatiza el procesamiento de información clínica y genera conocimiento útil a partir de los datos.

La plataforma desarrollada establece una base sólida para futuras mejoras, como incorporación de nuevos modelos de Machine Learning, ampliación de reportes, monitoreo en tiempo real e integración con fuentes externas de datos.

8. RECOMENDACIONES O TRABAJO FUTURO

Aunque el sistema desarrollado cumple con los objetivos definidos para el procesamiento, análisis y visualización de información clínica, existen oportunidades de mejora que permitirían ampliar su alcance, aumentar el desempeño y fortalecer sus capacidades analíticas.

8.1 Mejoras en el Proceso ETL

Se recomienda evolucionar el flujo ETL para soportar mayores volúmenes de información y automatizar aún más el procesamiento.

Posibles mejoras:

- Incorporar carga masiva de múltiples archivos simultáneamente.
- Permitir carga desde fuentes externas (API, Excel, almacenamiento cloud).
- Implementar validaciones avanzadas de calidad de datos.
- Programar ejecuciones automáticas mediante tareas periódicas.
- Generar bitácoras detalladas de errores durante la carga.

Beneficio esperado:

Mayor eficiencia operativa y reducción del procesamiento manual.

8.2 Evolución del Modelo de Machine Learning

Aunque el modelo actual alcanzó resultados satisfactorios (**Accuracy 91%**), se identifican oportunidades para mejorar especialmente métricas como Recall y F1-Score.

Mejoras propuestas:

- Incrementar el tamaño del dataset de entrenamiento.
- Balancear categorías de riesgo para reducir sesgo entre clases.
- Aplicar técnicas de validación cruzada (Cross Validation).
- Realizar optimización de hiperparámetros (Grid Search o Random Search).
- Evaluar modelos alternativos:
 - XGBoost
 - Gradient Boosting
 - LightGBM
 - Redes Neuronales

Beneficio esperado:

Mayor capacidad para detectar correctamente pacientes de riesgo y mejorar la calidad predictiva.

8.3 Mejoras en Reportes y Visualización

Se recomienda ampliar el módulo analítico para ofrecer una experiencia más completa al usuario.

Posibles mejoras:

- Dashboards interactivos en tiempo real.
- Comparativos históricos entre periodos.
- Reportes personalizados por usuario.
- Exportación en PDF y Excel.
- Visualización avanzada con más indicadores clínicos.

Beneficio esperado:

Mayor capacidad de análisis y mejor apoyo para la toma de decisiones.

8.4 Seguridad y Gobierno de Datos

Para fortalecer el entorno productivo se recomienda incorporar controles adicionales.

Mejoras propuestas:

- Implementar autenticación multifactor (MFA).
- Activar políticas avanzadas de acceso (RLS).
- Incorporar cifrado de información sensible.
- Registrar auditoría completa de operaciones.
- Implementar monitoreo y alertas.

Beneficio esperado:

Mayor protección de los datos y cumplimiento de buenas prácticas.

8.5 Escalabilidad y Arquitectura Cloud

El sistema puede evolucionar hacia una arquitectura más robusta y escalable.

Posibles mejoras:

- Contenerización mediante Docker.
- Despliegue automatizado (CI/CD).
- Balanceo de carga.
- Separación de ambientes (Desarrollo / QA / Producción).
- Integración con almacenamiento distribuido.

Beneficio esperado:

Mayor disponibilidad y facilidad de mantenimiento.

8.6 Nuevas Funcionalidades

Como evolución funcional del producto se recomienda incorporar:

- Módulo de alertas clínicas automáticas.
- Predicción en tiempo real desde formularios.
- Gestión de historial clínico.
- Comparación entre pacientes.
- Sistema de recomendaciones basado en IA.
- Integración con aplicaciones móviles.
- Panel administrativo avanzado.

El proyecto desarrollado constituye una base sólida para continuar evolucionando hacia una plataforma de análisis clínico más completa e inteligente.

Las mejoras propuestas permitirían aumentar la capacidad de procesamiento, fortalecer el desempeño del modelo de Machine Learning, enriquecer la experiencia del usuario y convertir el sistema en una solución más escalable, segura y orientada al análisis predictivo.